Infrastructure as Code Foundations

Part 1 — From cloud strategy to IaC architecture

SEAtS | 424812

University of Southampton

Intended Learning Outcomes

By the end of this session you will be able to:

1. Define cloud computing using NIST's five essential characteristics.
2. Compare public, private, community and hybrid deployment models, and on-premises vs cloud.
3. Distinguish the IaaS, PaaS and SaaS service models and common architectural patterns.
4. Explain how DevOps and "cattle not pets" thinking enable reproducible infrastructure.
5. Define Infrastructure as Code and capture its functional and non-functional requirements.
6. Decompose infrastructure into modules and compare declarative and imperative approaches.

© University of Southampton

Agenda

1. Cloud context: characteristics, deployment models, on-premises vs cloud.
2. Service models, architectural patterns, and "as-a-Service" offerings.
3. DevOps mindset: cattle vs pets, Development + Operations.
4. IaC definition, motivation, and requirements.
5. System architecture for IaC: modules, declarative vs imperative.
6. Summary.

© University of Southampton

What is Cloud Computing?

> “A model for enabling convenient, on-demand network access to a shared pool of configurable computing resources… that can be rapidly provisioned and released with minimal management effort or service provider interaction.” — NIST SP 800-145.

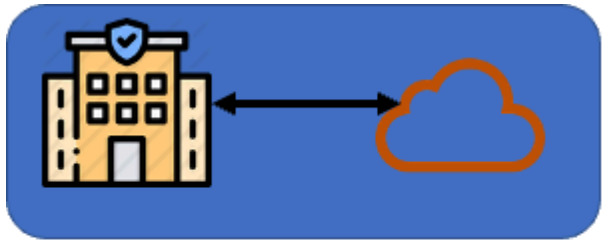
1. On-demand self-service
2. Broad network access
3. Resource pooling
4. Rapid elasticity
5. Measured service

- Improved security and “unlimited” capacity.
- Cost efficiencies; zero up-front capital.
- Elastic scaling as system grows.
- Time efficiencies via virtual resources.
- Power efficiency and visibility into consumption.
- Improved process control and quick recovery.

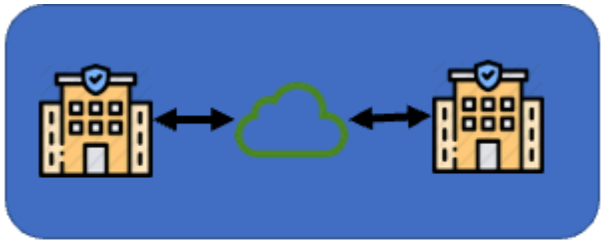
© University of Southampton

Cloud Deployment Models

1. **Public:** Infrastructure available to general public, owned by service provider.
2. **Private:** Dedicated to a single organisation, managed on/off premise.
3. **Community:** Shared by organisations with common concerns.
4. **Hybrid:** Combination of cloud types.



Private



Public



Hybrid



Community

© University of Southampton

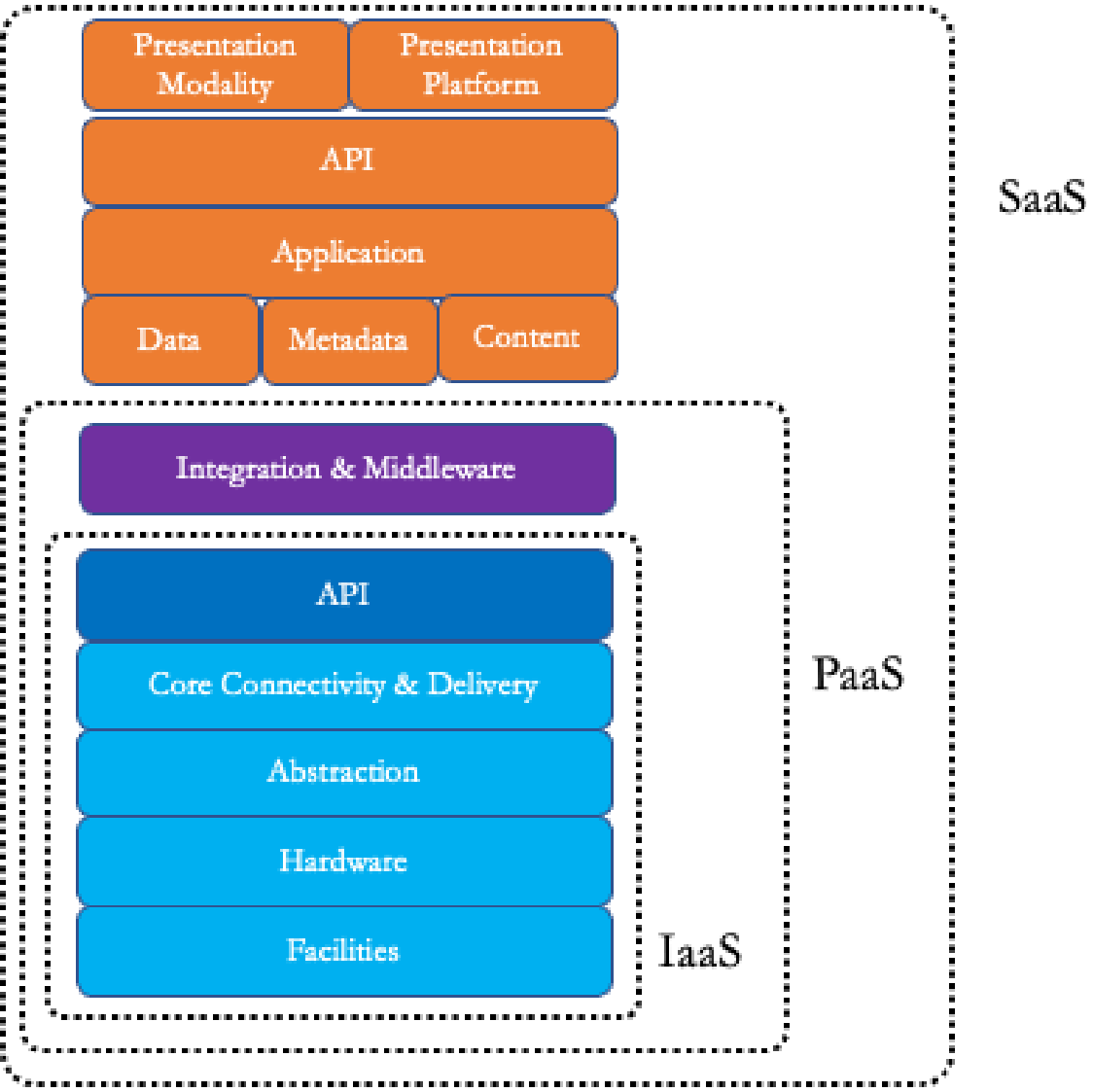
On premises Computing vs. Cloud Computing

On Premises	Cloud
Manually provisioned	Self-provisioned
Dedicated hardware	Shared hardware
Fixed capacity	Elastic capacity
Pay for capacity	Pay for use
CapEx + OpEx	Operational expenses
Managed via sysadmins	Managed via APIs



© University of Southampton

Main Service Delivery Models



IaaS: Provision virtual machines, networks, and storage inside the provider's infrastructure to run any OS or application stack.

- Control over guest OS and runtimes.
- Examples: EC2, Virtual Networks.

PaaS: Build and deploy custom applications using provider-supported runtimes and tooling.

- Auto elasticity and built-in CI/CD.
- Examples: App Engine, Azure App Service.

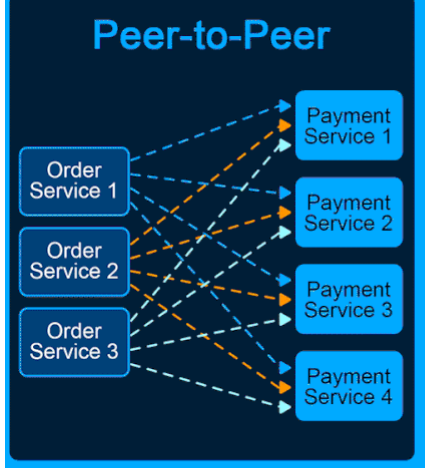
SaaS: Consume fully managed provider applications delivered over the cloud.

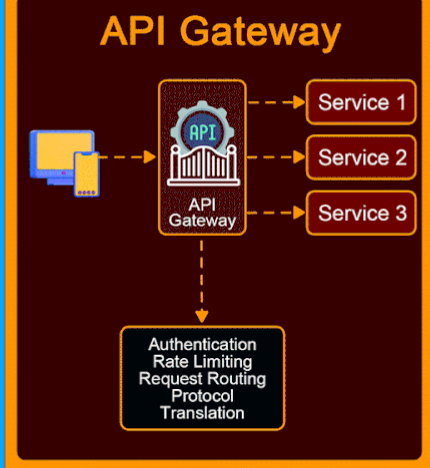
- No infrastructure management; pay per use.
- Examples: Gmail, Salesforce.

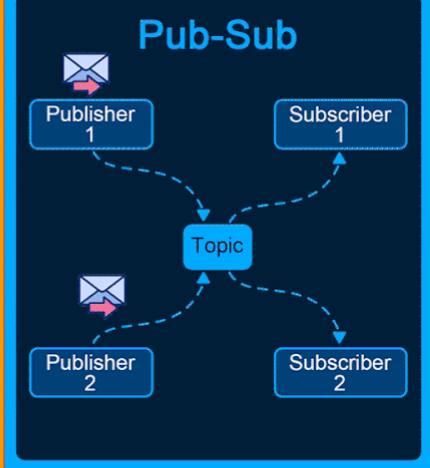
© University of Southampton

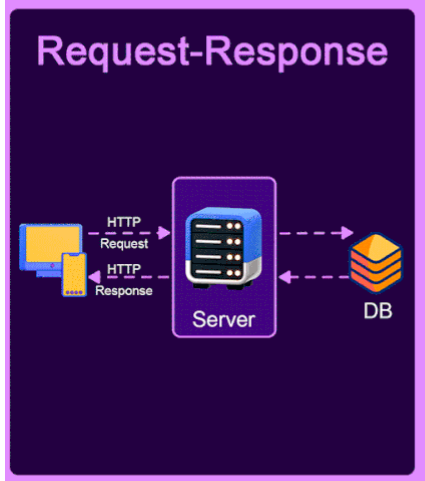
Architectural Patterns

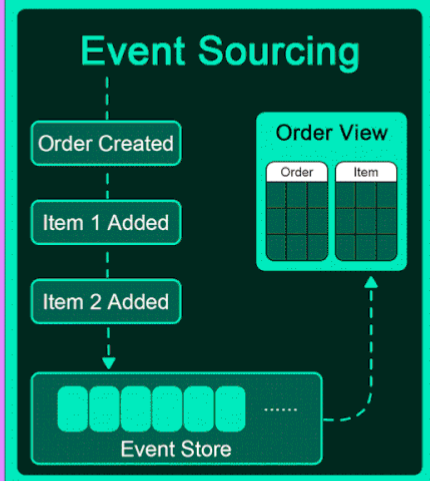
Pattern	Description
Peer-to-Peer	Direct communication between two components without a central coordinator.
API Gateway	Single entry point for all client requests to the backend services of an application.
Pub-Sub	Decouples publishers from subscribers through a message broker.
Request-Response	Client sends a request to a server and waits for a response.
Event Sourcing	Store application state changes as a sequence of immutable events.
ETL	Gather data from sources, transform it, and load into a destination.
Batching	Accumulate data over time/threshold before processing as a group.
Streaming Processing	Continuously ingest, process, and analyse data in real time.
Orchestration	Central coordinator manages interactions between distributed components.

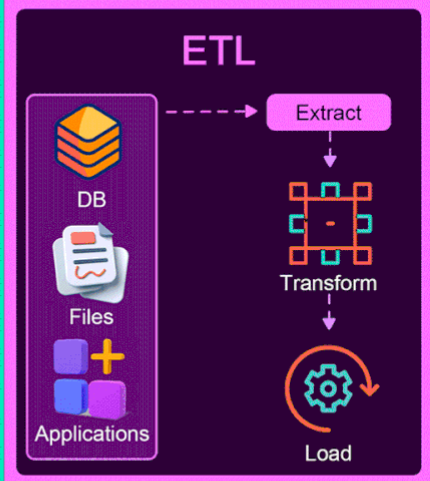


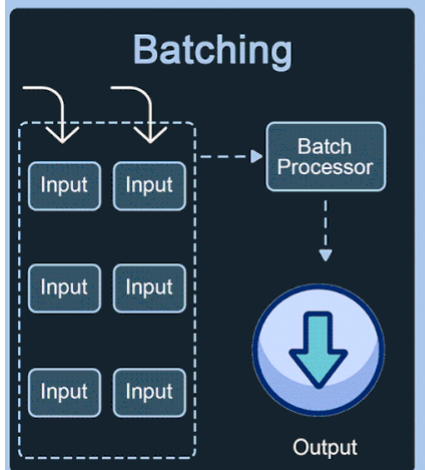


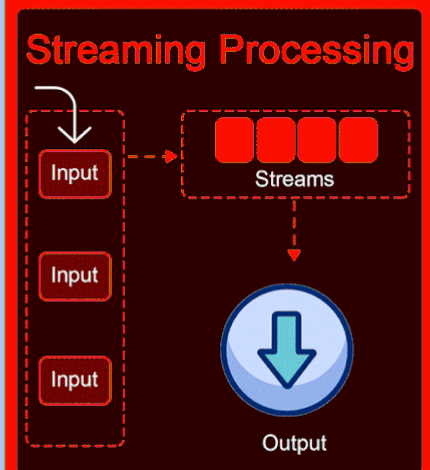


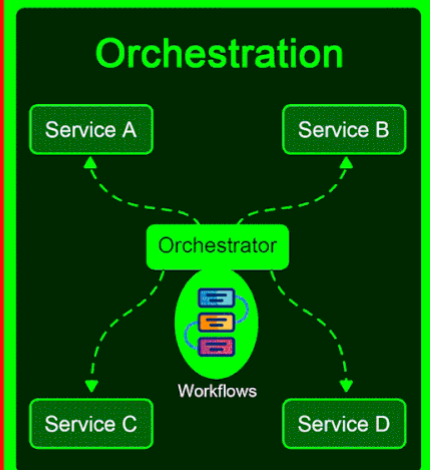












© University of Southampton

Many “as-a-Service” Offerings

Examples include Analytics, API, AI, Backend, Banking, Big Data, Blockchain, Business Process, Code, Content, Construction, Container, Crane, Edge Compute, Contact Centre, Communications Platform, Data, Desktop, Drone, Database, Distribution, Energy, Engagement, EV, Function, Farming, Firewall, Games, Governance, Hadoop, Hardware, Housing, Identity, Image, IoT, IT, Knowledge, Load Balancing, Logging, Management, Microgrid, Mobility, Monitoring, Metal, Mobile Backend, Machine Learning, Network, Network Defense, Payments, Policy, Product, Push Notification, QA, RAN, Recovery, Recycling, Robot, Search, Security, Solar, Shared Storage, Transportation, Testing, Time Series DB, Unified Communications, and “Anything as a Service.”

© University of Southampton

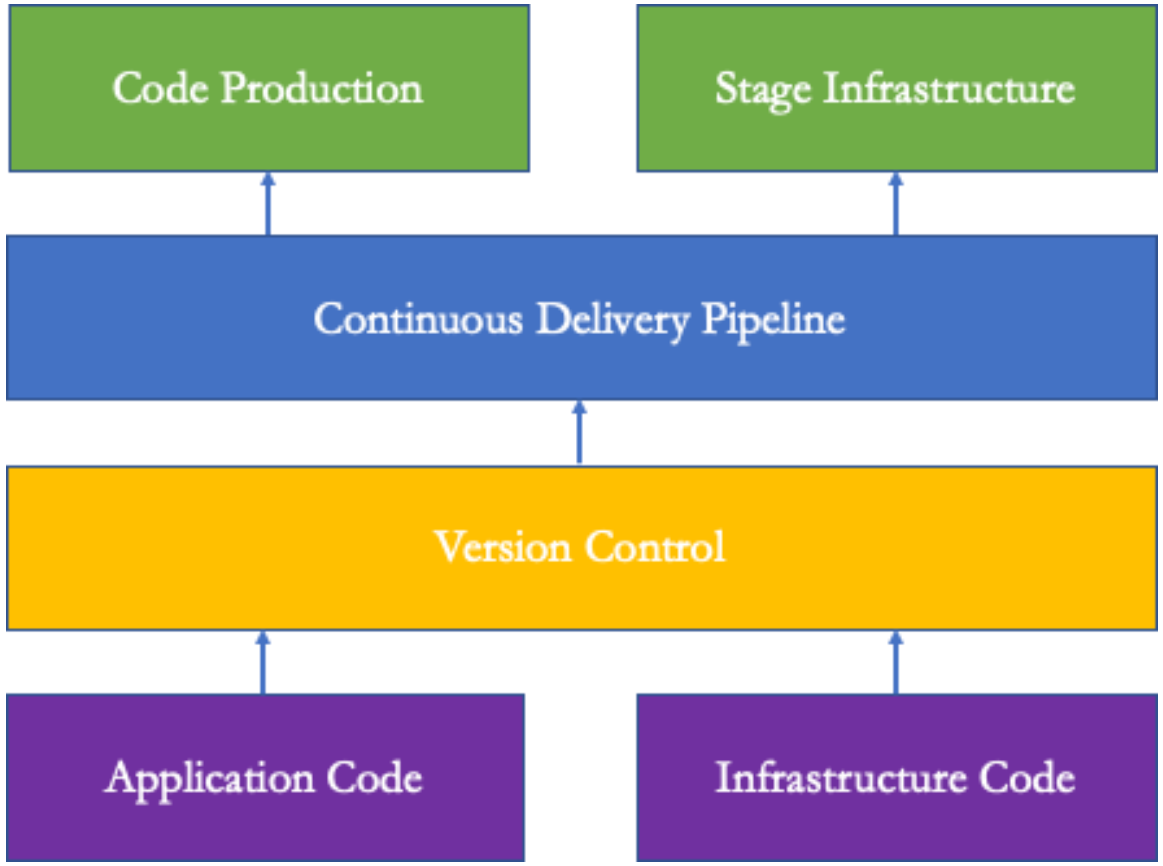
Infrastructure: Cattle vs Pets

Development environments should be consistent and reproducible, like managing “cattle” (identical, disposable instances) rather than unique “pets” (customised desktops). This ensures faster onboarding, predictable testing, and reliable automation.



© University of Southampton

DevOps =



Development + Operations

Development view: Focus on delivering features quickly with automated builds, tests, and infrastructure definitions.

Operations view: Focus on reliability, monitoring, capacity planning, and incident response.

DevOps bridge: Shared tooling (CI/CD, IaC), common metrics (SLOs), and collaborative culture align both perspectives.

© University of Southampton

IaC Definition & Motivation

Infrastructure as Code (IaC): Define and manage infrastructure via machine-readable files instead of manual provisioning.

Why essential?

- Rapid provisioning for software, hardware, and learning integration.
- Reduces configuration drift and enables CI/CD pipelines.
- Enhances repeatability, testing, and formal specification.
- Critical for distributed cloud systems.

© University of Southampton

Requirements in IaC

Functional examples: Deploy VPC/VNet; automate Kubernetes clusters; create IAM roles.

Non-functional examples: Scalability, security hardening, fault tolerance, cost-efficiency, auditability, compliance.

IaC formalises both sets of requirements in code.

- Cloud provider selection (AWS, Azure, GCP).
- State storage (S3, Blob, GCS).
- Secrets management (Vault, Secrets Manager).
- Access patterns (SSH, API gateways, private endpoints).
- Compliance constraints (residency, logging).
- Integration requirements (CI, monitoring, networks).

Methods: stakeholder interviews, architecture diagrams, threat modelling.

© University of Southampton

System Architecture for IaC

IaC encourages decomposition into modules:

- Networking (VPC, subnets, firewalls).
- Compute (VMs, containers).
- Storage (S3, RDS).
- Identity (IAM roles, policies).
- Monitoring (metrics, logging).

Declarative vs Imperative

Approach	Description	Examples
Declarative	Describe desired state; engine reconciles	Terraform, CloudFormation, Bicep
Imperative	Execute procedural steps	Ansible, Pulumi, scripts

Declarative scales better for large multi-component systems.

© University of Southampton

Summary

- Cloud computing delivers on-demand, elastic, pay-per-use resources (NIST's five characteristics).
- Deployment models (public, private, community, hybrid) and service models (IaaS, PaaS, SaaS) set what you control and what the provider manages.
- DevOps and "cattle not pets" demand consistent, disposable, reproducible environments.
- IaC defines infrastructure in machine-readable files — reducing drift and enabling CI/CD.
- Capture both functional and non-functional requirements, then decompose into modules (network, compute, storage, identity, monitoring).
- Declarative IaC describes the desired state and scales better than imperative step-by-step scripts.

© University of Southampton